

BÁO CÁO KỸ THUẬT & KIỂM THỬ HỆ THỐNG

BÀI TẬP 1: SỬA LỖI HARDCODE URL TRONG RESTTEMPLATE CỦA ORDER-SERVICE

Giải cứu API Khuyến mãi khỏi nguy cơ nghẽn cổ chai và sập dây chuyền trong hệ thống Microservices VietMart

Khóa học: Microservices System Design (Rikkei Education)	Học viên / Tác giả: Kỹ sư Phát triển Hệ thống
Hệ thống: VietMart E-Commerce Platform	Công nghệ: Spring Boot, Netflix Eureka, Spring Cloud LoadBalancer, JUnit 5, Mockito

1. Bối cảnh Nghiệp vụ & Hiện trạng Sự cố

Trong hệ thống thương mại điện tử **VietMart**, **order-service** đóng vai trò nòng cốt phục vụ tiếp nhận và xử lý đơn hàng. Để hiển thị thông tin sản phẩm và tính toán giá tiền trên màn hình chi tiết, **order-service** cần gọi sang **product-service** thông qua giao thức HTTP REST. Toàn bộ các dịch vụ được đăng ký và phân giải địa chỉ tự động qua **Netflix Eureka Service Registry**.

Một thực tập sinh đã triển khai lớp **ProductServiceClientRT** như dưới đây. Code chạy được trên máy tính cá nhân nhưng lập tức ném lỗi khi đưa lên môi trường Staging/Production:

```
// ProductServiceClientRT.java -- HIỆN TRẠNG ĐANG GẶP LỖI
@Component
public class ProductServiceClientRT {
    // LỖI 1: Không dùng @LoadBalanced RestTemplate -- không qua được Eureka
    private final RestTemplate restTemplate = new RestTemplate();

    public ProductInfo getById(Long productId) {
        // LỖI 2: Hardcode IP:port thay vì dùng service-id
        String url = "http://192.168.1.45:8082/api/products/{id}";

        // LỖI 3: Không có timeout -- thread có thể bị block mãi mãi
        return restTemplate.getForObject(url, ProductInfo.class, productId);
    }
}
```

2. Phân tích Chuyên sâu 3 Lỗi Kỹ thuật & Hậu quả Thực tế

2.1. Lỗi 1: Tự khởi tạo `new RestTemplate()` và thiếu annotation `@LoadBalanced`

- Bản chất kỹ thuật:** Toán tử `new RestTemplate()` tạo đối tượng không được quản lý bởi Spring ApplicationContext (Unmanaged Object). Quan trọng hơn, việc thiếu `@LoadBalanced` làm mất đi bộ chặn `LoadBalancerInterceptor`. Bộ chặn này chịu trách nhiệm chặn URL logic, truy vấn danh sách instance từ Eureka và phân phối request theo thuật toán cân bằng tải phía client (Client-side Load Balancing).
- Hậu quả Production:** Nếu cấu hình URL thành `http://product-service/...`, RestTemplate thông thường sẽ gọi qua DNS mạng và lập tức quăng lỗi `UnknownHostException`. Hệ thống hoàn toàn không thể cân bằng tải giữa nhiều node dù hạ tầng có mở rộng thêm 10-20 server.

2.2. Lỗi 2: Hardcode IP:Port vật lý (`http://192.168.1.45:8082`) thay vì dùng Service ID

- Bản chất kỹ thuật:** Vi phạm nguyên tắc **Location Transparency**. Trong môi trường Container (Docker/Kubernetes), địa chỉ IP và Port được cấp phát động ngẫu nhiên mỗi khi Pod/Container tái tạo hoặc co giãn.
- Hậu quả Production:**
 - Lỗi môi trường (Environment Incompatibility):** IP `192.168.1.45` chỉ là mạng LAN nội bộ máy dev, khi lên Cloud (IP `10.x.x.x`) sẽ gặp lỗi `Connection Refused` hoặc `Host Unreachable`.
 - Tạo Single Point of Failure (SPOF):** Toàn bộ lượng truy cập dồn vào 1 server vật lý duy nhất. Nếu máy này bảo trì hoặc sập nguồn, 100% người dùng VietMart không thể mua hàng.
 - Vô hiệu hóa Auto-Scaling:** Dù DevOps có nâng số lượng pod phục vụ Flash Sale lên gấp 10 lần, không một request nào đến được các pod mới.

2.3. Lỗi 3: Không cấu hình Timeout (`connectTimeout` & `readTimeout`)

Nguy cơ cực kỳ nghiêm trọng: Cạn kiệt luồng và Sập dây chuyền (Cascading Failure)!

- **Bản chất kỹ thuật:** Mặc định trong `SimpleClientHttpRequestFactory` của Spring, cả `connectTimeout` và `readTimeout` là **vô hạn (-1 / 0)**. Trong khi đó, Tomcat hoạt động theo mô hình **Thread-per-Request** với kích thước Thread Pool cố định (mặc định 200 worker threads).
- **Hậu quả Production:** Khi diễn ra sự kiện Khuyến mãi lớn (Flash Sale), `product-service` bị quá tải làm database bị khóa hoặc mạng chậm chạp. Toàn bộ thread gửi request từ `order-service` sang sẽ bị khóa cứng (Blocked/Waiting). Chỉ cần 200 lượt mua hàng bị treo, toàn bộ 200 thread Tomcat của `order-service` bị cạn kiệt (Thread Starvation). `order-service` sập hoàn toàn, kéo theo API Gateway nghẽn mạch (HTTP 504 Gateway Timeout), dẫn tới sụp đổ toàn bộ sàn VietMart.

3. Giải Pháp Khắc Phục Chuẩn Microservices

3.1. Bảng đối chiếu giải pháp

Tiêu chí	Mã nguồn có lỗi (Thực tập sinh)	Mã nguồn chuẩn hóa (Đã khắc phục)
Quản lý RestTemplate	<code>new RestTemplate()</code> độc lập	<code>@Bean @LoadBalanced</code> do Spring IoC quản lý tập trung
Địa chỉ gọi dịch vụ	Hardcode <code>192.168.1.45:8082</code>	Sử dụng Service ID logic <code>http://product-service</code>
Cân bằng tải	Không có (gọi cố định 1 IP)	Client-side Load Balancing (Round-Robin qua Eureka)
Kiểm soát Timeout	Vô hạn (Treo thread không thời hạn)	<code>connectTimeout = 2s</code> , <code>readTimeout = 3s</code>
Khắc phục sự cố Timeout	Crash hệ thống / Sập dây chuyền	Bắt <code>ResourceAccessException</code> → Trả về Fallback an toàn
Sản phẩm không tìm thấy	Ném lỗi không xác định	Bắt 404 → Ném <code>ProductNotFoundException</code>

3.2. Cấu hình Bean RestTemplate (`RestTemplateConfig.java`)

```
@Configuration
public class RestTemplateConfig {

    @Bean
    @LoadBalanced
    public RestTemplate restTemplate() {
        SimpleClientHttpRequestFactory factory = new SimpleClientHttpRequestFactory();
        factory.setConnectTimeout(Duration.ofSeconds(2)); // Kết nối tối đa 2s
        factory.setReadTimeout(Duration.ofSeconds(3));    // Đọc socket tối đa 3s
        return new RestTemplate(factory);
    }
}
```

3.3. Lớp Client Đã Sửa Đổi (`ProductServiceClientRT.java`)

```
@Component
@Slf4j
public class ProductServiceClientRT {

    private final RestTemplate restTemplate;
    private final String productServiceUrl;

    @Autowired
    public ProductServiceClientRT(
        @LoadBalanced RestTemplate restTemplate,
        @Value("${services.product-service.url:http://product-service}") String productServiceUrl
    ) {
        this.restTemplate = restTemplate;
        this.productServiceUrl = productServiceUrl;
    }

    public ProductServiceClientRT(RestTemplate restTemplate) {
        this(restTemplate, "http://product-service");
    }

    public ProductInfo getById(Long productId) {
        String url = productServiceUrl + "/api/products/{id}";
        log.info("Sending GET request to product-service: {}", url);
        try {
            return restTemplate.getForObject(url, ProductInfo.class, productId);
        } catch (ResourceAccessException e) {
            // LỖI TIMEOUT: Kích hoạt fallback để bảo vệ Tomcat Thread Pool
            log.error("ResourceAccessException (Timeout/Connection error): {}. Kích hoạt Fallback.", e.getMessage());
            return ProductInfo.fallback(productId);
        } catch (HttpClientErrorException.NotFound e) {
            // LỖI 404: Sản phẩm không tồn tại -> Ném ngoại lệ nghiệp vụ
            log.warn("Sản phẩm với ID {} không tồn tại (404 Not Found).", productId);
            throw new ProductNotFoundException(productId);
        }
    }
}
```

```
}  
}
```

4. Kết Quả Kiểm Thử Đơn Vị (Unit Test)

Sử dụng **JUnit 5** và **Mockito** để kiểm thử độc lập mà không cần khởi động Eureka Server thật:

Kịch bản Kiểm thử	Mục tiêu & Dữ liệu đầu vào	Kết quả mong đợi	Trạng thái
Test 1: Happy Path (<code>testGetById_Success</code>)	Mô phỏng RestTemplate trả về dữ liệu sản phẩm hợp lệ (ID: 101, Tên: "Sữa tươi VietMart", Giá: 35000).	Nhận đầy đủ DTO <code>ProductInfo</code> chính xác. RestTemplate được gọi đúng 1 lần.	PASSED ✔
Test 2: Timeout Simulation (<code>testGetById_Timeout_ReturnsFallback</code>)	Mô phỏng RestTemplate ném <code>ResourceAccessException("Read timed out after 3000ms")</code> .	Phương thức bắt ngoại lệ an toàn, trả về đối tượng <code>ProductInfo.fallback(id)</code> với status <code>UNAVAILABLE</code> .	PASSED ✔
Test 3: 404 Not Found (<code>testGetById_NotFound_ThrowsProductNotFoundException</code>)	Mô phỏng máy chủ trả về mã HTTP <code>404 Not Found</code> .	Bắt <code>HttpClientErrorException.NotFound</code> và ném đúng <code>ProductNotFoundException</code> .	PASSED ✔

Xác nhận kết quả: Toàn bộ 3 Unit Test đều vượt qua 100% (Pass). Mã nguồn tuân thủ nghiêm ngặt các nguyên lý Resilient Microservice Pattern.

5. Kết Luận

Việc chuyển đổi sang `@LoadBalanced RestTemplate` kết hợp thiết lập Timeout (2s/3s) và cơ chế Fallback đã giải quyết triệt để bài toán "ngheñ cồ chài" trong các chiến dịch khuyến mãi:

- Loại bỏ hoàn toàn sự phụ thuộc vào IP cứng, tương thích tuyệt đối với kiến trúc co giãn đám mây (Cloud Auto-Scaling).
- Ngăn chặn triệt để hiện tượng cạn kiệt luồng (Tomcat Thread Starvation), đảm bảo `order-service` luôn ổn định và không bao giờ xảy ra sập dây chuyền (Cascading Failure).